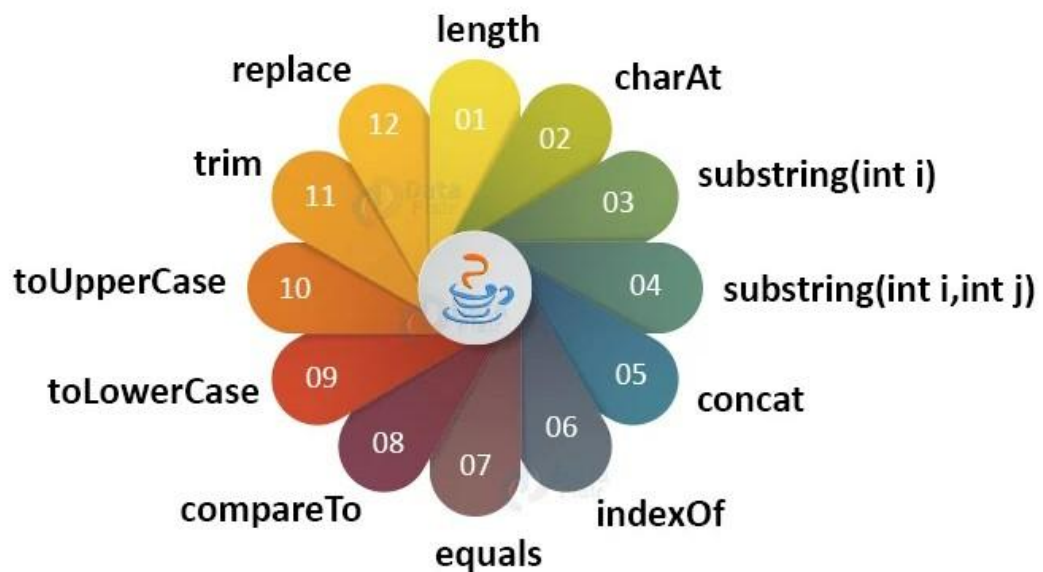


Java

String Functions :-

- The String class provides methods to manipulate and process strings.
- Strings are immutable, so most methods return a new string without changing the original.
- These functions help in comparison, case conversion, splitting, and extracting parts of a string.



- 1) **substring(int begin)** - Returns substring from given index to end.

Syntax –

```
str.substring(beginIndex);
```

- 2) **substring(int begin, int end)** - Returns substring between begin (inclusive) and end (exclusive).

Syntax –

```
str.substring(beginIndex, endIndex);
```

- 3) **trim()** - Removes leading and trailing spaces.

Syntax –

```
str.trim();
```

- 4) **toCharArray()** - Converts string into a character array.

Syntax –

```
char[] arr = str.toCharArray();
```

5) **split(String regex)** - Splits string into parts using regex (e.g., "\\s" for spaces)

Syntax –

```
String[] arr = str.split(regex);
```

6) **replaceAll(String regex, String replacement)** - Replaces all regex matches with given text.

Syntax –

```
str.replaceAll(regex, replacement);
```

Example –

```
package Basic_Concept;
```

```
import java.util.Arrays;
```

```
public class stringdemo {
```

```
    public static void main(String[] args) {
```

```
        String name=" Fo rtu ne Clo ud ";
```

```
        System.out.println("Substring: "+name.substring(7));
```

```
        System.out.println("Substring with Begin & End"+name.substring(3,7));
```

```
        System.out.println("Trim: "+name.trim());
```

```
        char arr[]=name.toCharArray();
```

```
        System.out.println("TocharArray: "+Arrays.toString(arr));
```

```
        String sarr[]=name.split("\\s");
```

```
        System.out.println("Split: "+Arrays.toString(sarr));
```

```
        System.out.println("Replace All: "+name.replaceAll("\\s","*"));
```

```
    }
```

```
}
```

Output –

Substring: ne Clo ud

Substring with Begin & End: rtu

Trim: Fo rtu ne Clo ud

ToCharArray: [, F, o, , r, t, u, , n, e, , C, l, o, , u, d,]

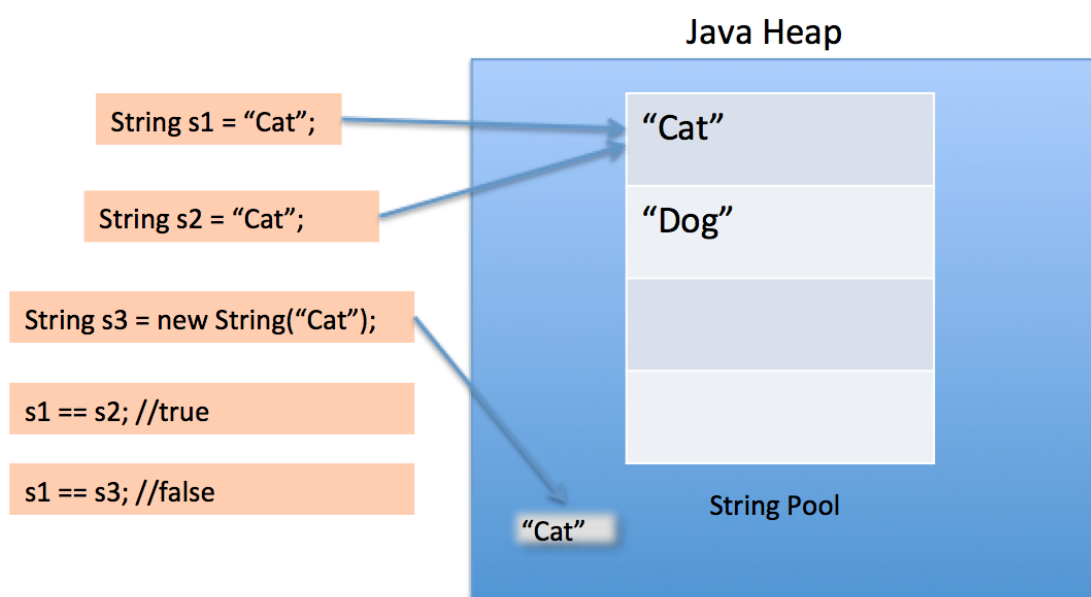
Split: [, Fo, rtu, ne, Clo, ud]

Replace All: *Fo*rtu*ne*Clo*ud*

Memory Allocation in String :-

Storage Process of String -

- When you write a string literal, JVM checks the String Constant Pool (SCP):
 - If it exists - reference is reused.
 - If not - new object is created in SCP.
- When you use new keyword, JVM always creates a new object in the heap, but the literal inside still goes to SCP.
- When you create a string, the Java Virtual Machine (JVM) can store it in two main memory areas:



1. String Constant Pool (SCP) inside Heap memory -

- When a string literal is created, JVM first checks the SCP.
- If the string already exists, its reference is reused; otherwise, a new one is created in SCP.
- This avoids duplicate objects and saves memory.
- Since Java 7, SCP is kept inside the Heap, and unused strings can be garbage collected.
- All strings in SCP are immutable.

Example:

```
String s1 = " Cat";  
String s2 = "Cat";  
  
// Both s1 and s2 point to the same object in SCP
```

Example Code –

package assignment;

public class string_task {

 public static void main(String[] args) {

 String name="Ramchandra";

 String name1="Ram";

 if(name.equals("Ram"))

 {

 System.out.println("Ramchandra is equals to Ram");

 }

 else

 {

 System.out.println("Ramchandra is not equals to Ram");

 }

 if(name==name1)

 {

 System.out.println("Ramchandra is equals to Ram");

 }

 else

 {

 System.out.println("Ramchandra is not equals to Ram");

 }

Output –

Ramchandra is not equals to Ram

Ramchandra is not equals to Ram

2. Heap (Outside SCP) -

- Strings created with the new keyword are stored in the Heap (outside SCP).
- Each new String("text") creates a fresh object, even if the same literal exists in SCP.
- So, two identical strings may exist — one in SCP and one in Heap.
- Heap string objects are not shared automatically like SCP strings.
- They are eligible for garbage collection when no reference points to them.
- This makes Heap storage less memory-efficient compared to SCP.

Example:

```
String s1 = new String("Cat");  
String s2 = new String("Cat");  
// s1 and s2 refer to different objects
```

Example Code -

```
package Basic_Concept;
```

```
public class stringclass_demo {
```

```
    public static void main(String[] args) {  
        String s1=new String ("Cloud");  
        String s2=new String ("Cloud");  
        if(s1.equals(s2))  
        {  
            System.out.println("Cloud is equals to Cloud");  
        }  
        else  
        {  
            System.out.println("Cloud is not equals to Cloud");  
        }  
  
        if(s1==s2)  
        {
```

```

        System.out.println("Cloud is equals to Cloud");
    }
    else
    {
        System.out.println("Cloud is not equals to Cloud");
    }
}
}

```

Output –

Cloud is equals to Cloud

Cloud is not equals to Cloud

Difference between Java Stack Memory and Heap Memory :-

Feature	Stack Memory	Heap Memory
Purpose	Stores local variables and call stack.	Stores objects and global variables.
Size	Smaller in size.	Larger in size.
Management	Automatic (LIFO).	Manual or garbage-collected.
Speed	Faster.	Slower.
Scope	Limited to function/method.	Accessible globally via references.
Thread Safety	Thread-safe (separate stack per thread).	Requires explicit synchronization.

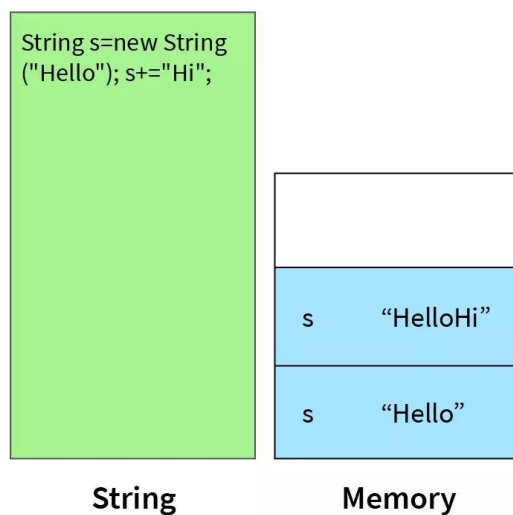
String Mutable Classes –

In Java, String mutable classes like StringBuffer and StringBuilder allow modification of string content without creating new objects, unlike the immutable String class.

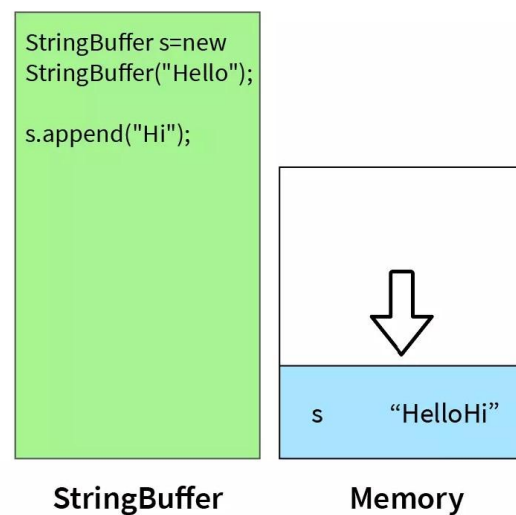
1) StringBuffer –

- StringBuffer is a mutable class in Java.
- It allows modification of strings without creating new objects.
- It is synchronized, meaning it is thread-safe.
- Operations like append(), insert(), delete(), reverse() modify the original object.

Immutable - cannot be changed



Mutable - can be changed



Syntax –

```
StringBuffer sb = new StringBuffer("YourText");
```

Example –

```
package Basic_Concept;
```

```
public class stringclass {
```

```
    public static void main(String[] args) {
```

```
StringBuilder str1=new StringBuilder("Good");

        System.out.println("Before Append: "+str1);

        System.out.println("Using Append: "+str1.append(" Students"));

        System.out.println("After Append: "+str1);

    }

}
```

Output –

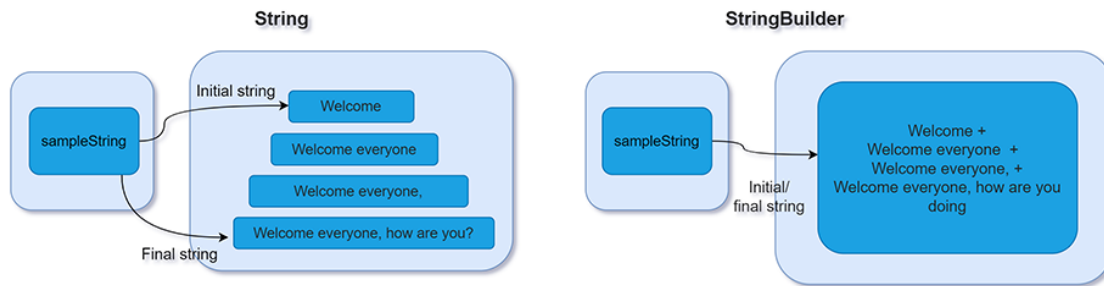
Before Append: Good

Using Append: Good Students

After Append: Good Students

2) StringBuilder –

- StringBuilder is also a mutable class in Java.
- It allows modification of strings without creating new objects.
- It is not synchronized, so it is faster than StringBuffer.
- Ideal for single-threaded programs.
- Supports same methods as StringBuffer: append(), insert(), delete(), reverse(), etc.



Syntax –

```
StringBuilder sb = new StringBuilder("YourText");
```

Example –

```
package Basic_Concept;
```

```
public class stringclass {
```

```
    public static void main(String[] args) {
```

```
        StringBuffer str2=new StringBuffer("Good");
```

```
        System.out.println("Before Append: "+str2);
```

```
        System.out.println("Using Append: "+str2.append(" Students"));
```

```
        System.out.println("After Append: "+str2);
```

```
    }
```

```
}
```

Output –

Before Append: Good

Using Append: Good Students

After Append: Good Students